Component Copier — User Guide

Component Copier is a Niagara Program Object that copies one or more components (points, folders, logic, extensions…) into one or many destinations in a single run. It can then make links to the new copies from a links CSV. Reverse undoes the last run.

Covers **N4 v3.3 / v3.4** and **AX v3.3-AX** (Niagara AX 3.5–3.8). The fields and actions are the same on both; see AX notes.

Contents

1. How it works in one minute
2. Every field, explained
3. Sources: whole folder or its contents
4. Destinations: the three modes
5. The links phase
6. Actions
7. Files it writes
8. Complete setups (recipes)
9. Results CSV: what each status means
10. Good habits
11. AX notes

---

1. How it works in one minute

```
componentSource (what)  ×  copyTo (where)   ──►  copy phase   ──►  log + results CSV
                                                      |
                  linksCsvPath (optional)   ──►  links phase  ──►  linker log + linker results CSV
```

- **Every source is copied to every destination.** 3 sources × 20 destinations = 60 copies.
- All the sources go to a destination together, like a Workbench multi-select paste: links between them are remade between the **copies**, not back to the originals.
- A destination that already has a component with that name is **skipped** — nothing is overwritten.
- It only runs when you press an action. Dry Run and Verify make no changes.

---

2. Every field, explained

Legend: **Set** = you set it · **Out** = the program writes it (read only). The display name (what the property sheet shows) is in brackets where it differs.

About

Field		What it is
`version`	Out	The program version.
`quickGuide`	Out	Short built-in help.

Last run

Field		What it is
`status`	Out	Live progress, then the summary (copied, skipped, failed, links made).

What to copy, and where

Field		Default	What it is
`destinationMode`	Set	BQL	**Direct / BQL / CSV** — how `copyTo` is read. See the three modes.
`componentSource` (Source(s) to Copy)	Set	blank	One source ord per line; `#` lines are ignored. A trailing `/*` copies a folder's **contents** instead of the folder — see Sources.
`copyTo` (Destination Target Location(s))	Set	blank	Direct: one destination. BQL: a query, each row a destination. CSV: a CSV file of destinations. The destination is the **container** that receives the copy (e.g. a device's `points` folder).
`keepAllLinks` (Keep All Incoming Links)	Set	false	Links coming into a source from **outside** what you're copying: **false** = left off the copy · **true** = the copy keeps them (it's linked to the same outside points as the original). Links inside the copied set are always kept, pointing at the copies.

Links after the copy

Field		Default	What it is
`linksCsvPath` (Autolinking Path)	Set	blank	Optional 5-column links CSV, run after the copy (same format as Linker). Blank = no links phase. With `componentSource` **and** `copyTo` both blank, the program runs **links only**.

Files and logs

Field		Default (blank =)	What it is
`logFilePath`	Set	`file:^logs/ComponentCopier.log`	Copy-phase log.
`resultsCsvPath`	Set	`file:^logs/ComponentCopier_results.csv`	One row per source × destination.
`linkerLogPath`	Set	`file:^logs/ComponentCopier_linker.log`	Links-phase log.
`linkerResultsCsvPath`	Set	`file:^logs/ComponentCopier_linker_results.csv`	One row per link.
`sampleCsvPath`	Set	`file:^logs/ComponentCopier_SAMPLE.csv`	Where **Create Sample CSV** writes the destinations CSV; the links sample goes beside it as `…_SAMPLE_links.csv`.
`maxArchives` (Max Archives to Keep)	Set	10	Old copies of each log / CSV kept (0–30; 0 = keep all).

All paths must be inside the station folder (`file:^…`).

---

3. Sources

Source line	What lands in the destination
`slot:/Templates/BXXXX_VAV/Logic`	The **folder itself** with everything inside: `dst/Logic/…`. Links inside it stay pointed at the new copy.
`slot:/Templates/BXXXX_VAV/Logic/*`	The folder's **contents**, each child copied on its own, no wrapper folder: `dst/Pid1`, `dst/Sub/…`. Links between them are remade afterwards (see the linker log).
`slot:/Templates/BXXXX_OAT`	That one component. `/*` on something that isn't a folder is ignored.

- Names can be typed as Workbench shows them (`VAV X-01` is escaped for you).
- A destination that **is** the source, or is inside it, is always skipped (no copying a folder into itself).

---

4. Destinations: the three modes

`destinationMode`	`copyTo` holds	Example
Direct	one container	`station:|slot:/Drivers/BacnetNetwork/BXXXXX_VAV_101/points`
BQL	a query — each row is a destination	`station:|slot:/Drivers|bql:select * from bacnet:BacnetPointDeviceExt where name = 'points'`
CSV	a CSV file	`file:^Csv/Destinations.csv`

Destinations CSV — one column, header row:

```
DestinationOrd
station:|slot:/Drivers/BacnetNetwork/BXXXXX_VAV_101/points
station:|slot:/Drivers/BacnetNetwork/BXXXXX_VAV_102/points
```

Test a BQL query in Workbench first and check how many rows it returns — that's how many copies you'll get of each source.

---

5. The links phase

`linksCsvPath` points at a 5-column CSV (header row required) — the same format as Linker:

```
BOrd1,Slot1,Direction,BOrd2,Slot2
station:|slot:/Drivers/BacnetNetwork/BXXXXX_OAT,out,>,station:|slot:/Drivers/BacnetNetwork/BXXXXX_VAV_101/points/Logic,in10
station:|slot:/Drivers/BacnetNetwork/BXXXXX_VAV_101/points/Logic,out,
<,station:|slot:/Drivers/BacnetNetwork/BXXXXX_VAV_101/points/DamperCmd,in16
```

- **Direction** `>` = link from 1 to 2 · `<` = link from 2 to 1.
- A blank slot means `out`.
- Both slots must exist; each link is read back after it's made. A link Niagara drops or can't activate (wrong slot, unresolved source, licence limit) is removed and reported as `ERROR`.
- A link that already exists is skipped.
- Links are named `link_<source>_<slot>_to_<targetSlot>_<number>` — the same names Linker and ForceRemove use, so all three tools recognise each other's links.

---

6. Actions

Action (slot name)	What it does
Execute (`execute`)	Copy, then make the links.
Dry Run - No Changes (`dryRun`)	Preview: lists every copy and link as `DRYRUN`.
Verify - Audit (`verify`)	Checks each destination for a component with the source's name (`FOUND` / `MISSING`) and each CSV link (`FOUND` / `MISSING` / `BROKEN` = there but not working). No changes.
Reverse Changes (`reverse`)	Undo the last Execute from its record: removes the links it made first, then the copies it placed. Anything changed since is skipped. Not a preview.
Cancel Request (`cancel`)	Stop after the current row.
Create Sample CSV (`createSampleCsv`)	Write starter destinations and links CSVs to `sampleCsvPath`.
Remove Old Logs (`pruneArchives`)	Apply `maxArchives` now.

7. Files it writes

File (in `Logs`)	Contents
`ComponentCopier.log`	Copy phase, step by step.
`ComponentCopier_results.csv`	Timestamp, SourceName, SourceSlotPath, DestinationName, DestinationSlotPath, Status, Message, Mode, KeepAllLinks, DurationMs.
`ComponentCopier_linker.log`	Links phase.
`ComponentCopier_linker_results.csv`	Timestamp, SourceName, SourceSlotPath, TargetName, TargetSlotPath, Status, Message, LinkMode, LinkName, DurationMs.
`ComponentCopier_<program name>_reverse.csv`	What the last Execute changed (used by Reverse).
`…_<date>_<time>.log / .csv`	Archives of earlier runs.

8. Complete setups

Placeholders throughout.

A. One logic folder into one device

Field	Value
destinationMode	Direct
componentSource	`station:|slot:/Templates/BXXXXX_VAV/Logic`
copyTo	`station:|slot:/Drivers/BacnetNetwork/BXXXXX_VAV_101/points`

B. A template's contents into every device's points folder (BQL)

Field	Value
destinationMode	BQL
componentSource	`station:|slot:/Templates/BXXXXX_VAV/*`
copyTo	`station:|slot:/Drivers/BacnetNetwork|bql:select * from bacnet:BacnetPointDeviceExt where name = 'points'`
keepAllLinks	false

Dry Run → check the row count in the results CSV → Execute.

C. Copy to a hand-picked list, then link (CSV + links)

1. **Create Sample CSV** → edit `ComponentCopier_SAMPLE.csv` (destinations) and `ComponentCopier_SAMPLE_links.csv` (links). Save them to `Csv/`.
2. `destinationMode` = CSV, `copyTo` = `file:^Csv/Destinations.csv`, `linksCsvPath` = `file:^Csv/Links.csv`.
3. Dry Run → Verify → Execute → Verify again (everything should be `FOUND`).

D. Links only (wire up existing components)

`componentSource` blank, `copyTo` blank, `linksCsvPath` = your CSV. Execute makes the links; Reverse removes them.

E. Several sources at once

```
# componentSource
station:|slot:/Templates/BXXXXX_VAV/Logic
station:|slot:/Templates/BXXXXX_VAV/Schedules
station:|slot:/Templates/BXXXXX_VAV/Graphics
```

All three land in each destination together; links between them point at the copies.

---

9. Results CSV

Status	Meaning
`COPIED`	Copied.
`LINKED`	Link made (linker results).
`DRYRUN`	Dry Run: would be done (or skipped — see Message).
`SKIPPED`	Not done; Message says why (already exists at destination, destination is the source, ...).
`FOUND` / `MISSING` / `BROKEN`	Verify results.
`REMOVED` / `DELETED`	Reverse took it away.
`FAILED` / `ERROR`	Failed; Message has the reason.

10. Good habits

- Back up the station. Dry Run, then Verify, then Execute.
- Check a BQL query's row count before Execute.
- Reverse before the next Execute if something went wrong (it undoes the last run only).
- Keep `copyTo` blank in the copy you keep as a template.

AX notes

- Program name **ComponentCopier4AX**, code `ComponentCopier4AX_v3.3.java`; AX 3.5–3.8.
- Same slots and actions; no imports to add.
- `maxArchives` is limited to 0–10. The AX version doesn't sort its own property sheet.